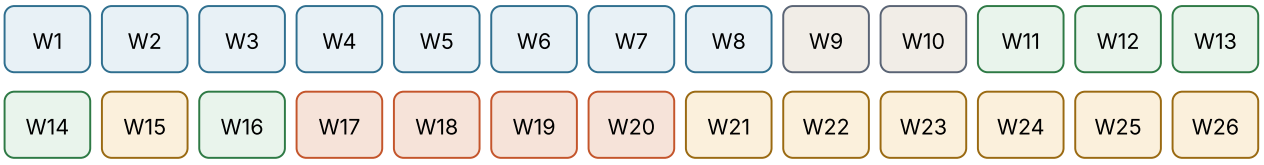
CLOUD SECURITY + BLUE TEAM · DOCUMENT 2 OF 2

The Plan

What to do every week for 26 weeks (26 October 2026 → 25 April 2027), with a step-by-step brief for each of the 13 projects and the capstone: build it, test it, detect attacks on it, write it up. Templates and checklists at the back.

Use with Document 1, The Roadmap · Version 1 · 24 October 2026



Blue: months 1–2 · Grey: holiday weeks · Green: months 3–4 · Amber W15: Security+ · Red: month 5 · Amber W21–26: month 6

Contents

0 How to use this plan
1 Month 1 · weeks 1–4
2 Month 2 · weeks 5–8
3 Month 3 · weeks 9–12
4 Month 4 · weeks 13–16
5 Month 5 · weeks 17–20
6 Month 6 · weeks 21–26
P Project briefs P1–P13 + capstone
T Templates
C Checklists and command reference
Q Interview questions by month

0 How to use this plan

> **Each week below lists what to do in each session. The steps for project work are in the project briefs (section P), so a line like "P3 steps 1–4" means "follow steps 1–4 of the P3 brief".**

The weekly slots

Day	Hours	Slot	Track
Mon	2.5	Cloud build	A/B
Tue	2.5	Theory + exam study	All
Wed	2.5	Blue-team lab	C
Thu	2.5	Cloud build	A/B
Fri	2	Practice: exam questions + TryHackMe/LetsDefend	All
Sat	6	Project day: finish, test, write up, push	All
Sun	0.5	Rest; plan the next week	—
Daily	0.3	**One alert** triaged on LetsDefend, two-line verdict in `notes/alert-log.md`	C

Days can swap (for example, if a class lands on Wednesday), but keep the balance: about two cloud builds, one blue-team lab, one theory session and one project day every week.

Every session

1. **Start (5 min):** read today's line; open `progress-log.md`.
2. **Work.** Stuck for 45 minutes? Write down what you tried and ask Claude.
3. **End (10 min):** two lines in the progress log; **destroy cloud resources**; check billing.

Every sprint (two weeks)

- **Start:** copy the sprint's "done when" list to the top of the progress log.
- **End (Saturday of the second week):** push to GitHub; quiz and explain-back session with Claude; a three-line retro: worked / didn't / change.

Repo layout you'll grow into

Path	Holds
`projects/P01-aws-baseline/` … `P13-sentinel/`, `capstone/`	One folder per project: `README.md` (the write-up), diagram, screenshots
`infra/aws/`, `infra/azure/`	Terraform
`detections/wazuh/`, `detections/aws/`, `detections/sentinel/`	Rules, each with a test note
`case-studies/`, `incident-reports/`	Investigations and IR reports
`playbooks/`	IR playbooks (month 5)
`notes/`	Alert log, interview bank, cheat sheets

REMEMBER

Redact before every push: account IDs, emails, access key IDs, public IPs, usernames and hostnames. Never commit secrets, Terraform state or `.tfvars` files.

1 Month 1 · Foundations + a secure baseline

Sprint 1 = weeks 1–2 · Sprint 2 = weeks 3–4 · Projects P1, P2 · Checkpoint CP1 on 22 Nov

Week 1 · 26 Oct – 1 Nov · "Open the account properly"

Day	Do
Mon	**P1 steps 1–3**: create the AWS account, root MFA, zero-spend and $5 budgets; choose eu-west-2
Tue	Theory: cloud service models, shared responsibility, regions and AZs. Security+: start your course (general security concepts). Make 20 flashcards.
Wed	Blue: learn the **triage method** (template T1). Create LetsDefend and TryHackMe accounts; start TryHackMe SOC Level 1; triage your first 3 alerts.
Thu	**P1 steps 4–6**: IAM Identity Center admin user with MFA; AWS CLI v2 on Kali; `aws configure sso`; `aws sts get-caller-identity`
Fri	20 Security+ practice questions; TryHackMe room; daily alert
Sat	**P1 steps 7–8**: CloudTrail trail to an S3 bucket; IAM Access Analyzer. Start the P1 write-up. Create `progress-log.md` with the Sprint 1 plan.

Week 2 · 2 – 8 Nov · "Least privilege + logs in motion"

Day	Do
Mon	IAM hands-on: write the S3 read-only policy (P1 step 9), attach it to a role, test it with the IAM policy simulator. Add an explicit deny and watch it win.
Tue	Theory: IAM model and policy evaluation; HTTP and APIs. Security+: threats, vulnerabilities, mitigations.
Wed	Blue: **P2 steps 1–3**: Wazuh syslog receiver, pfSense remote logging, prove logs arrive with `tcpdump`
Thu	**P1 step 10**: an EventBridge rule that emails you when the root user signs in. Test it once, then sign out of root.
Fri	Practice questions; daily alert
Sat	**Finish and push P1.** Sprint 1 review: quiz with Claude, retro.

Sprint 1 done when: P1 pushed · you can explain your policy line by line · pfSense logs reach Wazuh · 10+ alerts triaged.

Week 3 · 9 – 15 Nov · "Networks you can draw"

Day	Do
Mon	Build a VPC in the console: `10.40.0.0/16`, 2 public + 2 private subnets in two AZs, internet gateway, two route tables. Draw it.
Tue	Theory: cloud networking (subnets, routes, NAT, security groups vs NACLs, endpoints). Security+: network security.
Wed	Blue: **P2 steps 4–6**: SYN scan from Kali against your DMZ server; find the pfSense events; Ruleset Test; write a rule for "many blocks from one source"
Thu	Rebuild the same VPC with a Bash script of AWS CLI commands (`set -euo pipefail`), then delete it with a second script
Fri	Practice questions; TryHackMe (networking and Wireshark rooms)
Sat	**P2 steps 7–8**: tune and write up as Case 004. Write `docs/aws-vpc.md` with your diagram and scripts.

Week 4 · 16 – 22 Nov · "Instances without open doors" · CP1

Day	Do
Mon	EC2 in a public subnet with **no inbound rules** and the SSM role, reached through Session Manager. Then add a NACL deny and watch it break; fix it.
Tue	Theory: compute (instances, AMIs, EBS, metadata/IMDSv2). Security+: a timed practice test (aim for 60%+ now).
Wed	Blue: capture your own SYN scan in Wireshark on Kali and explain the packet pattern. Map P2 to MITRE T1046. TryHackMe SOC L1 progress.
Thu	Account hardening: require IMDSv2 for new instances; turn on EBS encryption by default; check S3 account-level Block Public Access is on
Fri	Practice; daily alert
Sat	**CP1 review** (Roadmap ch. 11). Push. **LinkedIn post #1:** "How I set up a secure AWS account" with redacted screenshots.

Sprint 2 done when: P2 pushed as Case 004 · VPC built three ways (console, script, diagram) · Session Manager working with no open ports · Security+ about 25%.

2 Month 2 · Workloads, data + web defence

Sprint 3 = weeks 5–6 · Sprint 4 = weeks 7–8 · Projects P3, P4, P5 · Checkpoint CP2 on 20 Dec

Week 5 · 23 – 29 Nov · "Storage locked down"

Day	Do
Mon	**P3 steps 1–3**: private S3 bucket, versioning, SSE-KMS with your own key, a bucket policy that denies non-HTTPS requests
Tue	Theory: cryptography, KMS envelope encryption, TLS and certificates. Security+: cryptography and PKI.
Wed	Blue: **P4 steps 1–3**: Nginx reverse proxy in front of DVWA and Juice Shop on the DMZ server; Wazuh agent reads the Nginx access log
Thu	**P3 steps 4–5**: CloudFront distribution with Origin Access Control and HTTPS redirect
Fri	Practice; daily alert
Sat	**P3 steps 6–7**: WAF web ACL (managed rules + rate limit) and WAF logging. Start the P3 write-up.

Week 6 · 30 Nov – 6 Dec · "Attack it, see it"

Day	Do
Mon	**P3 step 8**: test your own site: the bucket URL is denied, an injection-style query string is blocked by WAF, a rate-limit burst is throttled. Read the WAF logs.
Tue	Theory: the OWASP Top 10; web logs; common attacks. Security+: application security.
Wed	Blue: **P4 steps 4–6**: SQL injection and XSS against DVWA (low security) and Juice Shop from Kali; find the Wazuh web alerts; note what was missed
Thu	**P3 step 9**: list the CloudTrail events that would show someone making your bucket public; write them in the "Detect it" section
Fri	Practice; LetsDefend web-attack alerts
Sat	**Push P3 and P4.** Sprint 3 review and retro.

Sprint 3 done when: P3 and P4 pushed · you can explain OAC, WAF and SSE-KMS · you've seen your own attacks in Wazuh.

Week 7 · 7 – 13 Dec · "Everything as code"

Day	Do
Mon	Install Terraform on Kali. Rebuild the week 3 VPC in Terraform: `init` → `plan` → `apply` → `destroy`
Tue	Theory: infrastructure as code (state, drift, modules). Security+: architecture and cloud.
Wed	Blue: **P5 steps 1–3**: how email travels, reading headers, SPF/DKIM/DMARC checks with `dig`; TryHackMe phishing rooms
Thu	Rebuild P3 in Terraform (S3, KMS key, OAC, CloudFront, WAF). Move state to an encrypted S3 backend with locking.
Fri	Practice; LetsDefend phishing alerts
Sat	**P5 steps 4–6**: investigate three sample phishing emails safely; extract indicators; write the report

Week 8 · 14 – 20 Dec · "Consolidate" · CP2

Day	Do
Mon	Split your Terraform into a `modules/vpc` module; add `.gitignore`; a timed drill: build and destroy everything in under 30 minutes
Tue	*Optional:* AWS Cloud Practitioner exam. Otherwise a full Security+ practice test.
Wed	Blue: **P5 step 7**: write a phishing response playbook (template T4) and an "email security for small businesses" checklist
Thu	Terraform: EC2 in a **private** subnet with Session Manager through VPC endpoints (no NAT, no public IP)
Fri	Practice; daily alert
Sat	**CP2 review.** Push P5 and the Terraform. **LinkedIn post #2** (P3: "a website with no public bucket").

Sprint 4 done when: P5 pushed · all cloud work is in Terraform · Security+ about 50% · 40+ alerts triaged in total.

3 Month 3 · Identity + Windows

Sprint 5 = weeks 9–10 (holiday, about 10 h a week) · Sprint 6 = weeks 11–12 · Projects P6, P7 part 1 · CP3 on 17 Jan

Week 9 · 21 – 27 Dec · holiday week · "Build the Windows domain"

Session	Do
1 (3 h)	**P7 steps 1–3**: download Windows Server and Windows 11 evaluation images; add a CORP network (10.0.4.0/24) on pfSense with rules
2 (3 h)	**P7 step 4**: install Windows Server, set a static IP, promote it to a domain controller
3 (2 h)	Security+ light revision; daily alerts when you can

Week 10 · 28 Dec – 3 Jan · holiday week · "Make it visible"

Session	Do
1 (3 h)	**P7 steps 5–6**: Windows 11 joined to the domain; users, groups and an OU structure
2 (3 h)	**P7 steps 7–8**: Sysmon with a community config; Wazuh agents on both machines; Sysmon channel collected
3 (2 h)	Sprint 5 review: a short write-up of the domain build; plan for January

Week 11 · 4 – 10 Jan · "Secure login"

Day	Do
Mon	**P6 steps 1–3**: DynamoDB table, Lambda function with a least-privilege role, HTTP API, all in Terraform
Tue	Theory: authentication and authorization (MFA, OAuth 2.0, OIDC, JWT). Security+: identity and access management.
Wed	Blue: Windows event IDs (4624 logon types, 4625, 4688, 4720, 4728/4732, 4740) and Sysmon IDs 1, 3, 11, 13, 22. Cause each one deliberately; find it in Wazuh.
Thu	**P6 steps 4–5**: Cognito user pool with MFA required; JWT authorizer on the API
Fri	Practice; **book Security+ for 3–6 February**
Sat	**P6 steps 6–8**: tests (no token, bad token, good token), throttling, logging; write-up

Week 12 · 11 – 17 Jan · "Guardrails" · CP3

Day	Do
Mon	AWS Organizations: create the organization and a second "sandbox" account; an SCP that blocks stopping CloudTrail and limits regions. Test it from the sandbox account.
Tue	Theory: policy evaluation with SCPs and permission boundaries; identity in the cloud. Security+ practice test (aim 75%+).
Wed	Blue: **P7 step 9**: Group Policy for advanced auditing, command-line logging (4688) and PowerShell script block logging (4104); verify in Wazuh
Thu	A permission boundary experiment: an admin-like role that still can't escape its boundary. IAM Access Analyzer external-access findings.
Fri	Practice; daily alert
Sat	**CP3.** Push P6 and the P7 build notes. **LinkedIn post #3** (P6 secure API).

Sprint 6 done when: P6 pushed · SCP tested · Windows and Sysmon logs in Wazuh · Security+ practice 80%+ and the exam booked.

4 Month 4 · Detection + posture

Sprint 7 = weeks 13–14 · Sprint 8 = weeks 15–16 · Projects P7 part 2, P8, P9 · **Security+** · CP4 on 14 Feb

Week 13 · 18 – 24 Jan · "My SIEM sees my cloud"

Day	Do
Mon	**P8 steps 1–4**: a read-only identity for Wazuh, the AWS module in `ossec.conf`, outbound 443 from MGMT; CloudTrail events appear in Wazuh
Tue	Theory: the anatomy of a CloudTrail event; observability. Security+: security operations.
Wed	Blue: **P7 steps 10–12**: simulate (in your own domain) password guessing, a new user added to an admin group, an encoded PowerShell command and a scheduled task; check the alerts; write missing rules
Thu	**P8 steps 5–6**: generate the five test events in your own account; write the five Wazuh rules
Fri	Security+ practice; daily alert
Sat	**P8 steps 7–8**: test each rule (true and false positive); write-up

Week 14 · 25 – 31 Jan · "Managed detection"

Day	Do
Mon	Enable GuardDuty (trial starts); generate sample findings; read one finding's JSON field by field
Tue	Security+: two full timed practice exams; review every wrong answer
Wed	Blue: **P7 step 13**: a threat hunt with a hypothesis ("PowerShell downloaded something") using Sysmon events; build a MITRE ATT&CK coverage map of all your detections
Thu	**P9 steps 1–2**: AWS Config recorder; Security Hub with the CIS AWS Foundations and AWS Foundational Security Best Practices standards
Fri	Security+ practice
Sat	Push P7 (detections) and P8. Sprint 7 review.

Sprint 7 done when: P7 and P8 pushed · 10+ tested detections across Linux, Windows, network and AWS · coverage map in the repo.

Week 15 · 1 – 7 Feb · "Exam week"

Day	Do
Mon–Tue	Security+ revision of your weakest domains only; light flashcards
Wed–Fri	**Security+ exam** (on the date you booked); rest the evening before
Sat	Light: **P9 step 3**: install Prowler in a Python virtual environment on Kali and run a first scan

Week 16 · 8 – 14 Feb · "Assess like a consultant" · CP4

Day	Do
Mon	**P9 steps 4–5**: triage the findings (real risk, false positive, accepted); fix the top 10 in Terraform
Tue	**Start AWS Solutions Architect Associate study:** choose one course and one practice-exam set; read the exam guide
Wed	Blue: containers in brief: push a small image to ECR with scan on push; read the vulnerability report; compare it with Wazuh's vulnerability detection on Ubuntu
Thu	**P9 step 6**: write the posture report (template T5) with an executive summary
Fri	SAA practice; daily alert
Sat	**CP4.** Push P9. Update the CV. **LinkedIn post #4** (Security+ passed + P8).

Sprint 8 done when: Security+ passed · P9 pushed with a before/after score · SAA study started.

5 Month 5 · Incident response

Sprint 9 = weeks 17–18 · Sprint 10 = weeks 19–20 · Projects P10, P11 · CP5 on 14 Mar · applications start

Week 17 · 15 – 21 Feb · "Automate the first response"

Day	Do
Mon	**P10 steps 1–2**: an EventBridge rule for GuardDuty findings → SNS email; test with a sample finding
Tue	Theory: the incident response lifecycle (NIST SP 800-61 Rev. 3); detection and response in the cloud. SAA study.
Wed	Blue: write three playbooks with template T4: SSH/RDP brute force, phishing, malware on an endpoint
Thu	**P10 steps 3–5**: the isolation Lambda (Python): isolation security group, EBS snapshots, tags, notification
Fri	SAA practice; daily alert
Sat	**P10 steps 6–7**: Terraform it; the tag guardrail; test against your own tagged instance

Week 18 · 22 – 28 Feb · "Credentials and evidence"

Day	Do
Mon	**P10 step 8**: a second Lambda for IAM findings that deactivates the access key and attaches a deny-all policy
Tue	Theory: digital forensics basics (order of volatility, chain of custody, timelines; disk, memory and log evidence). SAA.
Wed	Blue: one CyberDefenders or Blue Team Labs investigation; write it up as a case study
Thu	Write `playbooks/cloud-ir.md`: your cloud containment steps for an instance, a key and a bucket
Fri	SAA practice
Sat	Push P10. Sprint 9 review. **Send your first 5 job applications.**

Sprint 9 done when: P10 pushed with both automations tested · four playbooks written · applications started.

Week 19 · 1 – 7 Mar · "Simulations 1 and 2"

Day	Do
Mon	**P11 simulation 1** (leaked key), steps 1–3: set up and act out the "attacker" in your sandbox account
Tue	SAA study (resilient architectures); interview practice on IR questions
Wed	**Simulation 1**, steps 4–6: detect (Wazuh, CloudTrail), respond, build the timeline
Thu	**P11 simulation 2** (public bucket): set up, detect, respond
Fri	SAA practice; daily alert
Sat	Write IR reports 1 and 2 (template T3)

Week 20 · 8 – 14 Mar · "Simulation 3" · CP5

Day	Do
Mon	**P11 simulation 3** (compromised EC2): trigger a GuardDuty test finding safely; watch P10 isolate the instance
Tue	SAA study (secure architectures)
Wed	Simulation 3 forensics: snapshot evidence, a timeline from CloudTrail and Flow Logs, rebuild from Terraform
Thu	IR report 3; a lessons-learned page across all three simulations
Fri	SAA practice; 5 more applications
Sat	**CP5.** Push P11. **LinkedIn post #5** ("I ran three cloud incidents end to end").

Sprint 10 done when: three IR reports with timelines pushed · applications going out every week.

6 Month 6 · DevSecOps, Azure + capstone

Sprint 11 = weeks 21–22 · Sprint 12 = weeks 23–24 · Sprint 13 = weeks 25–26 · Projects P12, P13, capstone · **AWS SAA** · CP6 on 25 Apr

Week 21 · 15 – 21 Mar · "Pipeline guards"

Day	Do
Mon	**P12 steps 1–2**: a GitHub Actions workflow with `terraform fmt`, `validate` and a Checkov scan
Tue	Theory: CI/CD and DevSecOps. SAA study.
Wed	Blue: KQL basics on Microsoft Learn; practise queries on Microsoft's free demo data
Thu	**P12 steps 3–4**: OIDC from GitHub to AWS with a tightly scoped role; `terraform plan` on pull requests
Fri	SAA practice exam; applications
Sat	**P12 step 5**: a pull request with an insecure change that the pipeline blocks; write-up

Week 22 · 22 – 28 Mar · "No secrets, no surprises"

Day	Do
Mon	**P12 steps 6–7**: apply on merge with manual approval; secret scanning with gitleaks
Tue	SAA: full practice exam 1, then review
Wed	Blue: SOAR concepts (compare with your P10 automations); LetsDefend; mock interview with Claude (SOC scenario)
Thu	Capstone design: architecture diagram, threat model (STRIDE), cost estimate
Fri	SAA practice; applications
Sat	Push P12. Sprint 11 review.

Week 23 · 29 Mar – 4 Apr · "Second cloud, second SIEM"

Day	Do
Mon	**P13 steps 1–2**: Azure for Students, secured; Entra ID users and groups; an RBAC test
Tue	SAA study
Wed	**P13 steps 3–4**: Log Analytics + Microsoft Sentinel; Azure Activity connector; the AWS CloudTrail connector
Thu	**P13 step 5**: three KQL analytics rules; test them with events you create
Fri	SAA practice exam 2; applications
Sat	**P13 step 6**: write-up, then delete the workspace to stop charges

Week 24 · 5 – 11 Apr · "Capstone build"

Day	Do
Mon	**Capstone steps 1–2**: network and data tiers in Terraform
Tue	SAA: weak areas
Wed	**Capstone step 3**: app tier and front door (CloudFront + WAF + load balancer)
Thu	**Capstone step 4**: logging, detections and auto-response wired in
Fri	SAA practice; applications
Sat	Capstone: the pipeline deploys it end to end. Sprint 12 review.

Week 25 · 12 – 18 Apr · "Capstone incident"

Day	Do
Mon	**Capstone step 5**: run the multi-stage incident scenario
Tue	SAA: final practice exam (aim 80%+)
Wed	Capstone: investigate, contain, recover; build the timeline
Thu	Capstone report + README + diagram
Fri	Record a 3–5 minute demo video; applications
Sat	Push the capstone; mock interview with Claude (cloud security scenario)

Week 26 · 19 – 25 Apr · "Finish line" · CP6

Day	Do
Mon–Tue	SAA light revision
Wed–Thu	**AWS Solutions Architect – Associate exam**
Fri	Portfolio polish: README highlights, a project index, the CV updated with all projects and certs
Sat	**CP6 review. LinkedIn post #6** (capstone demo). Write Roadmap version 2 for May – Dec 2027.

Sprint 13 done when: capstone pushed with a video · SAA passed · CV and LinkedIn current · applying every week.

P1 Secure AWS account baseline

Month 1 · Track B · Weeks 1–2 · Folder `projects/P01-aws-baseline/`

> **Objective: an AWS account set up as a security team would want it before any workload exists: protected root, named admin access with MFA, cost alarms, audit logging and an alert when the most powerful identity is used.**

Build

1. **Create the account** on the free plan. Pick **eu-west-2 (London)** as your home region and use it for everything except global services.
2. **Root user:** add MFA (an authenticator app or a security key). Store the root password in a password manager. From now on, root is for emergencies only.
3. **Billing → Budgets:** a *zero-spend budget* and a *monthly cost budget of $5*, both emailing you at 80% and 100%.
4. **IAM Identity Center:** enable it; create a user for yourself; require MFA; create a permission set using `AdministratorAccess`; assign it to your user for this account. Sign in through the Identity Center portal from now on.
5. **AWS CLI v2** on Kali (official installer). Check with `aws --version`.
6. **Connect the CLI** without long-term keys:

```
aws configure sso                    # start URL from Identity Center; name the profile
"lab"
aws sso login --profile lab          # opens the browser to log in
aws sts get-caller-identity --profile lab
echo 'export AWS_PROFILE=lab' >> ~/.bashrc
```

7. **CloudTrail:** create a trail for all regions writing to a new S3 bucket (Block Public Access on, encryption on), with **log file validation** enabled.
8. **IAM Access Analyzer:** create an account analyzer (external access). Note any findings.
9. **A least-privilege policy:** a role that can only read one bucket. Explain each line in the write-up.

```
{ "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:ListBucket", "s3:GetObject"],
    "Resource": ["arn:aws:s3:::BUCKET", "arn:aws:s3:::BUCKET/*"] }] }
```

10. **Root sign-in alert:** an SNS topic with your email subscribed, and an EventBridge rule that matches root console sign-ins and sends them to the topic. Console sign-in events usually arrive in **us-east-1**, so create the rule there.

```
{ "detail-type": ["AWS Console Sign In via CloudTrail"],
  "detail": { "userIdentity": { "type": ["Root"] } } }
```

Test it

- Sign in as root once; the email arrives (it can take a few minutes). Sign out.
- `aws s3 ls` with the read-only role works for that bucket and fails for others.
- CloudTrail event history shows your actions under your Identity Center name, not a shared account.

Detect it

Which events matter here? Root sign-in, `StopLogging` / `DeleteTrail`, `CreateUser` / `CreateAccessKey`, budget alerts. You'll turn these into SIEM rules in P8.

REMEMBER

Deliverables: README with a checklist of controls (✓) and why each matters; screenshots (MFA on, budgets, trail settings, alert email), redacted. **Cost:** about $0 (CloudTrail's first trail of management events is free; small S3 cost).
Talking point: "Before I built anything, I protected root, removed long-term keys, and made sure every action was logged and root use was alerted."

P2 Network threat detection

Month 1 · Track C · Weeks 2–4 · Becomes Case 004 · Folder `projects/P02-network-detection/`

> **Objective: firewall logs in the SIEM, and an alert when someone scans your network: the reconnaissance step of most attacks (MITRE ATT&CK T1046, Network Service Discovery).**

Build

1. **Wazuh receives syslog.** Back up, then add a second `<remote>` block to `/var/ossec/etc/ossec.conf` (keep the agent block):

```
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak-p2
sudo nano /var/ossec/etc/ossec.conf
  <remote>
    <connection>syslog</connection>
    <port>514</port>
    <protocol>udp</protocol>
    <allowed-ips>10.0.3.1</allowed-ips>
  </remote>
sudo systemctl restart wazuh-manager
```

2. **pfSense sends logs:** Status → System Logs → Settings → Remote Logging: source address MGMT, server `10.0.3.10:514`, contents "Firewall Events". Check that blocked traffic is logged (the default block rule logs by default).
3. **Prove logs arrive** before blaming Wazuh (the pipeline lesson from Case 001): `sudo tcpdump -ni any udp port 514 -c 5` on the Wazuh server.
4. **Generate the activity (your own lab only):** from Kali, `nmap -sS -p 1-1000 10.0.2.10`. Most ports are blocked by pfSense, so you get many block events from one source.
5. **Find it:** in Wazuh, search `rule.groups: pfsense`. If nothing decodes, copy one raw log line into **Ruleset Test**. If the format isn't recognised, set pfSense's log message format to BSD (RFC 3164) and test again.
6. **Write the correlation rule** if the built-in rules don't already alert on volume. Put it in `local_rules.xml`, using the pfSense block rule's ID (from Ruleset Test) as the parent:

```
<group name="local,pfsense,recon,">
  <rule id="100110" level="10" frequency="20" timeframe="60" ignore="120">
    <if_matched_sid>PARENT_ID</if_matched_sid>
    <same_source_ip />
    <description>pfSense: 20+ blocked connections from one source in 60 s (possible port
scan)</description>
    <mitre><id>T1046</id></mitre>
  </rule>
</group>
```

7. **Test and tune:** a true positive (the scan fires it), a false-positive check (normal browsing from Kali doesn't), and a slow-scan test (`nmap -T1`) to find the blind spot, as in Case 002.
8. **Write up as Case 004** with template T2, including the blind spot and what you'd add.

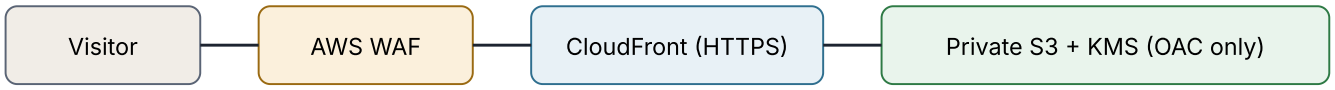
REMEMBER

Deliverables: Case 004, rule in `detections/wazuh/`, screenshots of the alert and Ruleset Test.
Talking point: "I proved logs reached the server with tcpdump before touching the SIEM, then wrote and tested a correlation rule, including its blind spot."

P3 Secure static website (S3 + CloudFront + WAF)

Month 2 · Track A/B · Weeks 5–7 · From the video · Folder `projects/P03-secure-website/`

> **Objective: publish a website whose bucket is never public, served only over HTTPS through a CDN, and protected by a web application firewall whose logs you can read.**



Build (console first in weeks 5–6, then Terraform in week 7)

1. **Bucket:** Block Public Access on (all four settings); Object Ownership "bucket owner enforced" (ACLs off).
2. **Versioning** on; default encryption **SSE-KMS** with a customer managed key you create (alias `alias/p3-site`).
3. **Deny non-HTTPS access** in the bucket policy:

```
{ "Sid": "DenyInsecureTransport", "Effect": "Deny", "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::BUCKET", "arn:aws:s3:::BUCKET/*"],
  "Condition": { "Bool": { "aws:SecureTransport": "false" } } }
```

4. **CloudFront** with the bucket as origin and **Origin Access Control**. Apply the bucket policy statement CloudFront gives you: it allows the `cloudfront.amazonaws.com` service only when `AWS:SourceArn` is your distribution. Because the bucket uses KMS, also allow CloudFront to use the key (`kms:Decrypt`) in the **key policy**, with the same condition.
5. Viewer protocol policy **Redirect HTTP to HTTPS**; default root object `index.html`; upload a simple page.
6. **WAF:** a web ACL for CloudFront (it must be created in **us-east-1**) with the AWS managed *Core rule set* and *Known bad inputs*, plus a **rate-based rule** (e.g. 100 requests per 5 minutes per IP for testing). Attach it to the distribution.
7. **WAF logging** to an S3 bucket whose name starts with `aws-waf-logs-` (a requirement), or to CloudWatch Logs.
8. **Test your own site:**

```
curl -I https://BUCKET.s3.eu-west-2.amazonaws.com/index.html   # expect 403: no direct
access
curl -I http://dxxxx.cloudfront.net/                            # expect 301 → https
curl -I "https://dxxxx.cloudfront.net/?id=1%27%20OR%201=1--"     # expect 403 from WAF
for i in $(seq 1 150); do curl -s -o /dev/null -w "%{http_code}\n"
https://dxxxx.cloudfront.net/; done | sort | uniq -c
```

Then find the blocked requests in the WAF logs: which rule matched, the client IP, the URI. The rate rule can take a minute or two to react.

9. **"Detect it" section:** the CloudTrail events that would show the bucket being exposed (`PutBucketPolicy`, `DeletePublicAccessBlock`, `PutBucketAcl`) and the WAF signals worth alerting on (spikes in blocks, rate-rule triggers).

REMEMBER

Deliverables: diagram, Terraform in `infra/aws/p3-site/`, the four test results, WAF log excerpt.
Cost: the WAF web ACL and rules have monthly fees charged by the hour. Destroy after testing.
Talking point: "The bucket is private and encrypted with my own key; only my CloudFront distribution can read it, over HTTPS, behind a WAF whose logs I analysed."

P4 Web attack detection

Month 2 · Track C · Weeks 5–6 · Folder `projects/P04-web-attack-detection/`

> **Objective: see application-layer attacks (SQL injection, XSS) in your SIEM by logging every request to your vulnerable apps. They stay in the DMZ, never in the cloud.**

Build

1. **Move the containers behind Nginx.** Recreate DVWA and Juice Shop so they listen only on localhost on new ports (e.g. `127.0.0.1:4281` and `127.0.0.1:3001`). Install Nginx on the Ubuntu host: `sudo apt install nginx`.
2. **Reverse proxy on the original ports**, so pfSense rules don't change. In `/etc/nginx/sites-available/labapps` (then link it into `sites-enabled` and run `sudo nginx -t && sudo systemctl reload nginx`):

```
server { listen 4280; location / { proxy_pass http://127.0.0.1:4281; proxy_set_header Host $host; } }
server { listen 3000; location / { proxy_pass http://127.0.0.1:3001; proxy_set_header Host $host; } }
```

Nginx runs on the host, so **ufw applies to it**, unlike Docker-published ports. Allow only Kali: `sudo ufw allow from 10.0.1.101 to any port 3000,4280 proto tcp`.

3. **Wazuh agent reads the access log.** Add to the agent's `ossec.conf` and restart the agent:

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/nginx/access.log</location>
</localfile>
```

4. **Attack DVWA** from Kali (security level "low"): the SQL injection page with `1' OR '1'='1`, and the reflected XSS page with `<script>alert(1)</script>`.
5. **Attack Juice Shop:** the login form with an injection-style email (`' OR 1=1--`), and a search with a script tag.
6. **Compare:** which requests raised Wazuh web-attack alerts (group `web`/`attack`), and which didn't? **POST bodies are not in access logs**, so login-form injection is invisible here. That's a real-world blind spot: note it and how a WAF or app logging would close it. Write one custom rule for a pattern you saw that wasn't flagged.

REMEMBER

Deliverables: write-up with a table of each attack → logged? → alerted? → why; the Nginx config; screenshots. **Talking point:** "I learned that access logs miss POST bodies, so login injection needs WAF or app-level logging. Knowing your log source's limits is part of detection."

P5 Phishing investigation

Month 2 · Track C · Weeks 7–8 · Folder `projects/P05-phishing/`

> **Objective: investigate suspicious emails the way a SOC analyst does: headers, authentication results, links and attachments, indicators, verdict, response.**

Learn

1. **How email travels:** sender's mail server → (relays) → receiver's mail server. Each hop adds a `Received:` header; read them **bottom to top**.
2. **The key headers:** `From` (what the user sees, easily faked), `Return-Path` (bounce address), `Reply-To`, `Message-ID`, `Authentication-Results` (SPF, DKIM and DMARC verdicts).
3. **Check a domain's records:**

```
dig +short TXT example.com              # SPF: which servers may send for the domain
dig +short TXT _dmarc.example.com       # DMARC: what to do when checks fail
(none/quarantine/reject)
dig +short TXT SELECTOR._domainkey.example.com   # DKIM public key (selector is in the DKIM-
Signature header)
```

Investigate (safely)

4. **Get samples:** LetsDefend phishing alerts and public collections of real phishing emails (e.g. the *phishing_pot* repository on GitHub). Work inside Kali.
5. **Safety rules:** open `.eml` files **as text only**; never click links or open attachments; defang indicators in notes (`hxxp://bad[.]example`); check URLs, domains and file hashes with reputation services (VirusTotal, URLscan.io) rather than visiting them.
6. **For each of three emails**, fill template T1 plus: sender analysis, SPF/DKIM/DMARC results, the lure (urgency, brand impersonation), indicators (sender IP, domains, URLs, attachment hashes), verdict, and actions (block sender/domain, search mailboxes for the same message, reset credentials if anyone clicked).
7. **Write the phishing playbook** (template T4) and a one-page "email security for small businesses" checklist: MFA, SPF/DKIM/DMARC at `p=reject` over time, link protection, user reporting button, training.

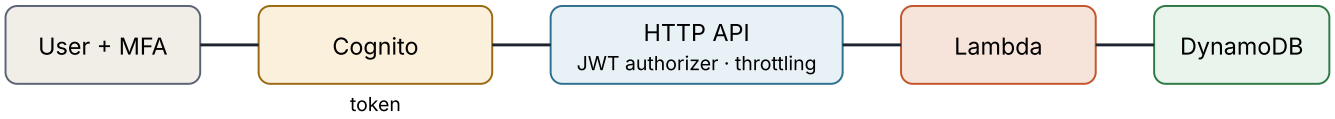
REMEMBER

Deliverables: three investigation reports, the playbook, the checklist. **Talking point:** "Phishing is the most common alert type. I can read headers, check SPF/DKIM/DMARC, extract indicators safely and decide what response is needed."

P6 Secure serverless API with Cognito

Month 3 · Track A/B · Weeks 11–12 · From the video · Folder `projects/P06-secure-api/`

> **Objective: a serverless API that only signed-in users with MFA can call, where the code has the minimum permissions, abuse is throttled, and every request is logged.**



Build (all in Terraform, in `infra/aws/p6-api/`)

1. **DynamoDB table** `notes` (partition key `user_id`, sort key `note_id`), on-demand billing, encryption on.
2. **Lambda** (Python) that reads and writes notes for the **calling user only**, taking the user ID from the token's `sub` claim that API Gateway passes in the request context, never from the request body.
3. **Execution role:** only `dynamodb:GetItem`, `PutItem` and `Query` on that one table's ARN, plus writing its own logs.
4. **Cognito user pool:** a strong password policy, **MFA required** (authenticator app), email verification; an app client. Create a test user and register an authenticator app for it through Cognito's managed login page.
5. **HTTP API** (API Gateway v2) with routes `GET /notes` and `POST /notes` and a **JWT authorizer**: issuer `https://cognito-idp.eu-west-2.amazonaws.com/POOL_ID`, audience = the app client ID.
6. **Throttling and logging:** default route throttling on the stage (e.g. burst 20, rate 10 per second); access logs to a CloudWatch log group.

Test it

7. Get a token (enable `USER_PASSWORD_AUTH` on the test client only for this; real apps use the hosted login or SRP):

```
aws cognito-idp initiate-auth --auth-flow USER_PASSWORD_AUTH --client-id CLIENT_ID \
  --auth-parameters USERNAME=testuser,PASSWORD='...'
# returns a SOFTWARE_TOKEN_MFA challenge → answer it with respond-to-auth-challenge and your
6-digit code
curl -i https://API_ID.execute-api.eu-west-2.amazonaws.com/notes                    #
expect 401
curl -i -H "Authorization: Bearer $TOKEN" https://API_ID.execute-api.eu-west-
2.amazonaws.com/notes   # expect 200
```

Also test: a token with one character changed (401), an expired token (401), a burst of 200 quick requests (some 429s). Read the access log for each case.

8. **Detect it:** what would show abuse? Spikes of 401s (token guessing), 429s (abuse), Cognito failed sign-ins, and changes to the authorizer or Lambda role in CloudTrail.

REMEMBER

Deliverables: diagram, Terraform, the five test results with log excerpts, a STRIDE table for the API. **Talking point:** "I used the user ID from the verified token, not from the request, so one user can't read another's notes. That's how broken access control, the top OWASP risk, gets prevented."

P7 Windows / Active Directory SOC lab

Months 3–4 · Track C · Weeks 9–14 · Folder `projects/P07-windows-ad-soc/`

> **Objective: build the environment most companies run (a Windows domain), make it visible in your SIEM, and detect the attacker behaviour SOCs see every day.**

Part 1: build (weeks 9–12)

1. **Images:** Windows Server and Windows 11 Enterprise **evaluation** ISOs from the Microsoft Evaluation Center (free, time-limited). Snapshot each VM after installing it.
2. **Network:** add a new VMware network adapter to pfSense, assign it as **CORP**, `10.0.4.1/24`.
3. **CORP rules (top to bottom):** allow CORP → Wazuh 10.0.3.10 TCP 1514/1515; allow CORP → CORP DC DNS; allow CORP → internet 80/443 (updates); **block** CORP → LAN, DMZ, MGMT (everything else). Add a temporary LAN Kali → CORP rule only when a test needs it, then disable it.
4. **Domain controller** (10.0.4.10, 4 GB RAM): static IP, rename, install the AD DS role, promote to a new forest, e.g. `corp.lab.internal`. DNS on the DC itself.
5. **Windows 11** (10.0.4.20): DNS pointing to the DC; join the domain.
6. **Directory:** OUs (Users, Workstations, Servers, Admins); a few users; a separate admin account for yourself (never browse as admin); a "Helpdesk" group.
7. **Sysmon** on both machines with a well-known community configuration (e.g. SwiftOnSecurity's): `sysmon64.exe -accepteula -i sysmonconfig.xml`.
8. **Wazuh agents** (MSI installer, manager `10.0.3.10`). Add the Sysmon and PowerShell channels to each agent's `ossec.conf`, then restart the agent (`NET STOP WazuhSvc`, `NET START WazuhSvc`):

```
<localfile><location>Microsoft-Windows-Sysmon/Operational</location>
<log_format>eventchannel</log_format></localfile>
<localfile><location>Microsoft-Windows-PowerShell/Operational</location>
<log_format>eventchannel</log_format></localfile>
```

9. **Group Policy for visibility:** Advanced Audit Policy (logon, account management, process creation, other object access); *Include command line in process creation events* (System → Audit Process Creation); *Turn on PowerShell Script Block Logging*. Run `gpupdate /force` and verify events appear in Wazuh.

Part 2: detect (weeks 13–14) · all actions in your own domain

#	Simulate	Expected evidence	MITRE
10a	Ten wrong passwords for a test user (e.g. `runas /user:CORP\testuser cmd`)	4625 ×10, maybe 4740 lockout	T1110
10b	On the DC: `net user simadmin Str0ng!Pass1 /add /domain`, then `net group "Domain Admins" simadmin /add /domain`	4720, 4728	T1136.002, T1098
11	A harmless encoded PowerShell command: `powershell -EncodedCommand` with "Write-Output hello" encoded in Base64	4688 with the command line, 4104 script block	T1059.001, T1027
12	`schtasks /create /tn LabTest /tr "cmd /c echo hi" /sc once /st 23:59`	4698 (needs Other Object Access auditing), Sysmon 1	T1053.005

For each: did Wazuh alert? At what level? If not, write a rule (template T2). Afterwards, **clean up**: delete `simadmin` and the task.

13. **Threat hunt:** hypothesis "PowerShell was used to download a file". Simulate `Invoke-WebRequest https://example.com -OutFile test.html`, then hunt Sysmon event 1 (command lines containing `Invoke-WebRequest`) and event 3 (network connections by `powershell.exe`). Write up the hunt even if you only find your own test. Then build a **coverage map** of all your detections in the MITRE ATT&CK Navigator and export it to the repo.

REMEMBER

Deliverables: build notes, the detection table with results, custom rules, the hunt write-up, the ATT&CK layer. **RAM:** pfSense + Kali + Wazuh + DC + Windows 11 is about 22 GB. **Talking point:** "I built an AD domain, turned on the right auditing, and tested detections for brute force, new admin accounts, obfuscated PowerShell and persistence."

P8 Hybrid cloud SIEM: CloudTrail → Wazuh

Month 4 · Track B/C · Weeks 13–14 · Folder `projects/P08-hybrid-siem/`

> **Objective: one SIEM watching your home lab and your AWS account, with five tested detections for the cloud attacks that matter most.**

Build

1. **A read-only identity for Wazuh:** an IAM user (or a role, if you later run Wazuh in AWS) whose policy allows only `s3:ListBucket` and `s3:GetObject` on the CloudTrail bucket, plus `kms:Decrypt` on the trail's key if it's encrypted. This is the one place a long-term key is acceptable here, so tag it, rotate it every 90 days, and alert on its misuse.
2. **Credentials on the Wazuh server:** in the root user's `~/.aws/credentials` under a profile named `wazuh`, file mode `600`. Install the Python dependencies listed in Wazuh's "Monitoring AWS-based services" prerequisites for your version.
3. **The AWS module** in `/var/ossec/etc/ossec.conf` (back it up first), then restart the manager:

```
<wodle name="aws-s3">
  <disabled>no</disabled>
  <interval>10m</interval>
  <run_on_start>yes</run_on_start>
  <skip_on_error>yes</skip_on_error>
  <bucket type="cloudtrail">
    <name>YOUR-CLOUDTRAIL-BUCKET</name>
    <aws_profile>wazuh</aws_profile>
  </bucket>
</wodle>
```

4. **Firewall:** pfSense must let MGMT reach the internet on TCP 443 (outbound only). Check with `curl -I https://s3.eu-west-2.amazonaws.com` from the Wazuh server. Then look for events: `rule.groups: amazon` or `data.aws.eventName: *`.

REMEMBER

Expect a delay. CloudTrail delivers log files every few minutes, and Wazuh checks every 10 minutes, so an event can take 15–30 minutes to appear. Note this in your write-up: detection latency is a real design question.

Detect: five rules (in `detections/aws/` and `local_rules.xml`)

#	Detection	Match on	MITRE
1	Root user activity	`aws.userIdentity.type` = Root	T1078.004
2	Console login without MFA	`aws.eventName` = ConsoleLogin and `aws.additionalEventData.MFAUsed` = No	T1078.004
3	New access key created	`aws.eventName` = CreateAccessKey	T1098.001
4	Security group opened to the world	`AuthorizeSecurityGroupIngress` and the log contains `0.0.0.0/0`	T1562.007
5	Logging tampered with	`StopLogging`, `DeleteTrail`, `UpdateTrail`	T1562.008

```
<group name="local,aws,cloudtrail,">
  <rule id="100205" level="12">
    <if_sid>PARENT_ID</if_sid>   <!-- the CloudTrail parent rule: find it with Ruleset Test
-->
    <field name="aws.eventName">^StopLogging$|^DeleteTrail$|^UpdateTrail$</field>
    <description>AWS: CloudTrail logging changed by $(aws.userIdentity.arn)</description>
    <mitre><id>T1562.008</id></mitre>
  </rule>
</group>
```

5. **Generate the events** in your own account: sign in as root once; a test IAM user without MFA signs in; create and delete an access key; open and close a test security group to `0.0.0.0/0` on port 22 (no instance attached); try `aws cloudtrail stop-logging` from the sandbox account. The SCP from week 12 should block it, but the **AccessDenied attempt is still logged**, which is a great detection.
6. **Write the rules,** each with a clear description that names who did it.
7. **Test each one:** true positive, plus a false-positive check (e.g. a security group opened to one IP must *not* fire rule 4).
8. **Write up:** the architecture, latency, the rule table with test results, and what you'd add next (GuardDuty findings, S3 data events).

REMEMBER

Talking point: "I connected CloudTrail to my on-prem SIEM through a least-privilege read-only identity and wrote five detections for the most common cloud attack steps. I even caught the attempt to stop logging that my SCP blocked."

P9 Cloud posture assessment

Month 4 · Track B · Weeks 14–16 · Folder `projects/P09-posture-assessment/`

> **Objective: assess an AWS account against recognised benchmarks, decide what really matters, fix it in code, and write a report a manager or client could act on.**

1. **AWS Config:** turn on the recorder (all resources in your region). Config charges per recorded configuration item; at lab scale it's small, but check billing after a day.
2. **Security Hub:** enable it with the **CIS AWS Foundations Benchmark** and **AWS Foundational Security Best Practices** standards. Wait for the first scores (it can take up to a day).
3. **Prowler** (open source) on Kali, in a Python virtual environment:

```
python3 -m venv ~/prowler-venv && source ~/prowler-venv/bin/activate
pip install prowler
prowler aws --profile lab                  # full scan; reports in ./output
prowler aws --list-compliance              # see available frameworks, then run one with --
compliance
```

4. **Triage every high and critical finding** into: *fix* (real risk), *accept* (documented reason, e.g. "lab account, no hardware MFA key"), or *false positive*. Record it in a findings table.
5. **Fix the top 10 in Terraform** (not by clicking), then re-scan and record the before/after score.
6. **The report** (template T5): executive summary in plain language, the score before and after, findings by severity, what was fixed, accepted risks with owners, and recommendations.

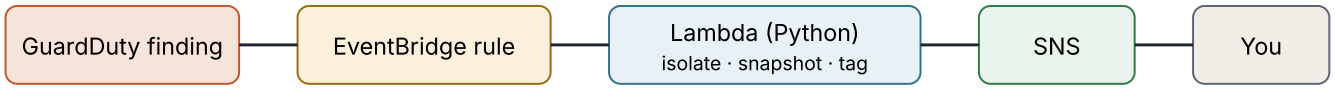
REMEMBER

Deliverables: the report (PDF or Markdown), the findings table, before/after screenshots, the Terraform fixes. **Cost:** Security Hub and Config have free trials and then charge per check or item. Disable them at the end of the month if you don't need them. **Talking point:** "I assessed my account against CIS, prioritised findings by real risk, fixed them as code, and reported the improvement." This is also the paid assessment service for small businesses.

P10 Automated threat response

Month 5 · Track B · Weeks 17–18 · From the video · Folder `projects/P10-auto-response/`

> **Objective: when GuardDuty reports a threat, respond in seconds: isolate the instance, preserve evidence, lock the stolen key and notify a human.**



1. **Notification first:** an SNS topic (email) and an EventBridge rule with this pattern targeting it:

```
{ "source": ["aws.guardduty"], "detail-type": ["GuardDuty Finding"],
  "detail": { "severity": [{ "numeric": [">=", 4] }] } }
```

2. Test with sample findings: `aws guardduty create-sample-findings --detector-id DETECTOR_ID`. Read one email and the JSON.
3. **An isolation security group** in your VPC with **no inbound and no outbound rules**.
4. **The isolation Lambda**, with environment variables `ISOLATION_SG` and `TOPIC_ARN`:

```
import boto3, os
ec2, sns = boto3.client("ec2"), boto3.client("sns")

def handler(event, context):
    f = event["detail"]
    iid = f.get("resource", {}).get("instanceDetails", {}).get("instanceId")
    if not iid:
        return {"skipped": "not an EC2 finding"}
    tags = ec2.describe_tags(Filters=[{"Name": "resource-id", "Values": [iid]},
                                      {"Name": "key", "Values": ["auto-isolate"]}])["Tags"]
    if not tags or tags[0]["Value"] != "true":           # guardrail: only opted-in
instances
        return {"skipped": iid}
    ec2.modify_instance_attribute(InstanceId=iid, Groups=[os.environ["ISOLATION_SG"]])
    ec2.create_snapshots(InstanceSpecification={"InstanceId": iid},
                         Description=f"IR evidence for finding {f['id']}")
    ec2.create_tags(Resources=[iid], Tags=[{"Key": "ir-status", "Value": "isolated"}])
    sns.publish(TopicArn=os.environ["TOPIC_ARN"], Subject="EC2 instance isolated",
                Message=f"{iid} isolated: {f['type']} (severity {f['severity']})")
    return {"isolated": iid}
```

5. **Least-privilege role** for the function: `ec2:DescribeTags`, `ec2:ModifyInstanceAttribute`, `ec2:CreateSnapshots`, `ec2:CreateTags`, `sns:Publish` on your topic, and its own CloudWatch logs.
6. **Terraform it all** (rule, function, role, security group, topic) in `infra/aws/p10-response/`.
7. **Test with your own instance:** launch a small instance tagged `auto-isolate=true`. Sample findings contain made-up instance IDs, so copy a sample finding's JSON, put your real instance ID in it, and send it as a **Lambda test event**. Check that the security group changed, snapshots exist, the tag is set and the email arrived.

8. **Second function for stolen keys:** for IAM-related findings, read `resource.accessKeyDetails` (user name and key ID), then `iam.update_access_key(..., Status="Inactive")` and attach the AWS managed `AWSDenyAll` policy to that user. Guardrail: act only on users in your sandbox account or tagged for it.

REMEMBER

A detail interviewers love: security groups are stateful, so swapping in an empty one may **not cut connections that are already open**. Mature playbooks also add a network ACL deny for the instance's subnet or stop the instance after evidence is taken. Mention this trade-off in your write-up.
Talking point: "My automation contains a threat in seconds, keeps the evidence, only touches resources that opted in, and always tells a human."

P11 Cloud incident simulations

Month 5 · Track B/C · Weeks 19–20 · Folder `incident-reports/` · run in your **sandbox** account

> **Objective: three realistic incidents, each worked through the full lifecycle (prepare, detect, analyse, contain, eradicate, recover, learn) and written up as a professional report.**

Simulation 1 · Leaked access key

1. **Prepare:** in the sandbox account, an IAM user `sim-dev` with read access to one test bucket only; create an access key for it. **Never post it anywhere**; the "leak" is only acted out.
2. **Act as the attacker** from Kali with that key: `aws sts get-caller-identity`, list buckets, read the test bucket, then try things it shouldn't be able to do (create a user, list EC2 instances, call another region). The failures matter: they're the "noisy" part of a real attack.
3. Note your start time. Now switch roles: you're the analyst.
4. **Detect and analyse:** find the activity in Wazuh (P8 rules) and CloudTrail. Filter by the access key ID; list every API call, source IP, region and error. Did the "attacker" create anything (persistence)?
5. **Contain and eradicate:** deactivate the key (or let P10 do it), attach a deny-all policy, check for new users, keys or roles, then delete the key and rotate anything it could read.
6. **Report** with template T3: a timeline from first use to containment, impact, root cause (how keys leak), and lessons (no long-term keys; alert on first use from a new IP).

Simulation 2 · Public S3 bucket

1. **Prepare:** a test bucket with **dummy data only** (fake "customer" CSV) and server access logging on.
2. **The mistake:** remove Block Public Access on that bucket and add a public-read policy (the classic misconfiguration). Fetch an object anonymously with `curl` to "be" the outsider.
3. **Detect:** Security Hub/Config finding, an Access Analyzer finding, `PutBucketPolicy` in Wazuh. How long did each take?
4. **Contain:** turn Block Public Access back on; remove the policy.
5. **Analyse exposure:** the server access logs show which objects were read, from which IPs and when. In real life this decides whether you must notify (GDPR/NDPA personal data).
6. **Report** plus a prevention section: account-level Block Public Access, an SCP, Checkov in the pipeline (P12).

Simulation 3 · Compromised EC2 instance

1. **Prepare:** a small instance tagged `auto-isolate=true`, with a simple web page; VPC Flow Logs on.
2. **Trigger a finding safely:** GuardDuty's documentation lists safe test methods, such as DNS lookups of special test domains that raise command-and-control findings. Run one from the instance through Session Manager.
3. **Watch P10 respond:** isolation, snapshots, the email. Record the times.
4. **Investigate:** the GuardDuty finding details, CloudTrail (who launched or changed the instance), Flow Logs (where it connected), and the snapshot (attach it to a separate forensic instance read-only and look at logs and recently changed files).
5. **Recover:** terminate the instance and redeploy from Terraform: a known-good rebuild, not cleanup.

6. **Report,** then a **lessons-learned page across all three**: what was detected fast or slow, what you'd automate next.

REMEMBER

Deliverables: three IR reports with timelines, evidence screenshots, a lessons-learned page.
Talking point: "I've run three cloud incidents end to end. For the leaked key, I traced every API call by key ID, checked for persistence, and documented the timeline from first use to containment."

P12 Secure infrastructure-as-code pipeline

Month 6 · Track A/B · Weeks 21–22 · Folder `.github/workflows/` + `projects/P12-secure-pipeline/`

> **Objective: every infrastructure change is formatted, validated, security-scanned and reviewed before it reaches AWS, using short-lived credentials with no stored keys.**

1. **Checks on every pull request:** `terraform fmt -check`, `terraform validate`, Checkov.
2. **Branch protection** on `main`: pull requests required, checks must pass.
3. **OIDC from GitHub to AWS:** create an IAM OIDC identity provider for `token.actions.githubusercontent.com` (audience `sts.amazonaws.com`), and a **plan role** (read-only) that trusts only your repository:

```
"Condition": {
  "StringEquals": { "token.actions.githubusercontent.com:aud": "sts.amazonaws.com" },
  "StringLike":   { "token.actions.githubusercontent.com:sub": "repo:OWNER/REPO:*" } }
```

4. **The workflow** (`.github/workflows/terraform.yml`). Pin each action to a current version when you set it up:

```
name: terraform
on:
  pull_request: { paths: ["infra/**"] }
permissions: { id-token: write, contents: read }
jobs:
  check-and-plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform -chdir=infra/aws fmt -check -recursive
      - run: terraform -chdir=infra/aws init -backend=false
      - run: terraform -chdir=infra/aws validate
      - uses: bridgecrewio/checkov-action@master   # pin a release tag
        with: { directory: infra/aws }
      - uses: aws-actions/configure-aws-credentials@v4
        with: { role-to-assume: "arn:aws:iam::ACCOUNT_ID:role/gh-terraform-plan", aws-
region: eu-west-2 }
      - run: terraform -chdir=infra/aws init && terraform -chdir=infra/aws plan -no-color
```

5. **Prove it works:** open a pull request that makes a bucket public or opens SSH to `0.0.0.0/0`. Checkov fails the check and the PR can't merge. Screenshot it. Then fix it in the same PR and watch it pass.
6. **Apply on merge, with approval:** a second job on `push` to `main` that uses a GitHub **environment** called `production` with a required reviewer (you), and a separate **apply role** whose trust policy only accepts `repo:OWNER/REPO:environment:production`.
7. **Secret scanning:** add gitleaks as a check, and turn on GitHub's own secret scanning. Test it with a fake key pattern in a throwaway branch, then delete the branch.

REMEMBER

Public repo caution: pull requests from forks don't get OIDC tokens, which is what you want. Keep the plan role read-only and the apply role behind the approval gate.

Talking point: "My pipeline uses OIDC with no stored keys, separate plan and apply roles, and blocks insecure Terraform before it can merge."

P13 Multi-cloud SIEM with Microsoft Sentinel

Month 6 · Track B/C · Week 23 · Folder `projects/P13-sentinel/`

> **Objective: get comfortable in Azure and Microsoft's SIEM, the stack many UK and European employers run, by bringing AWS and Azure logs into Sentinel and writing KQL detections.**

1. **Azure for Students** with your school email; MFA on; a budget alert; a resource group `rg-p13`.
2. **Entra ID:** two test users and a group; give the group `Reader` on the resource group; sign in as a test user and confirm what Reader can't do.
3. **Log Analytics workspace + Microsoft Sentinel** (check the current free-trial terms first). Connect **Azure Activity** (free).
4. **AWS connector:** follow Microsoft's guide "Connect Microsoft Sentinel to Amazon Web Services to ingest AWS service log data" (CloudTrail via S3 and SQS, with an AWS role that trusts Sentinel). *Entra ID sign-in logs need a P1/P2 licence; use them only if your tenant has a trial.*
5. **Three KQL analytics rules,** each tested with an event you create:

```
// 1. CloudTrail logging tampered with
AWSCloudTrail
| where EventName in ("StopLogging", "DeleteTrail", "UpdateTrail")
| project TimeGenerated, UserIdentityArn, SourceIpAddress, EventName, ErrorCode

// 2. AWS console login without MFA
AWSCloudTrail
| where EventName == "ConsoleLogin" and AdditionalEventData has "\"MFAUsed\":\"No\""

// 3. Azure role assignment created
AzureActivity
| where OperationNameValue =~ "MICROSOFT.AUTHORIZATION/ROLEASSIGNMENTS/WRITE"
| where ActivityStatusValue == "Success"
```

6. **Write up** (including how KQL compares with Wazuh rules), then **delete the workspace and the AWS connector resources** so nothing keeps billing.

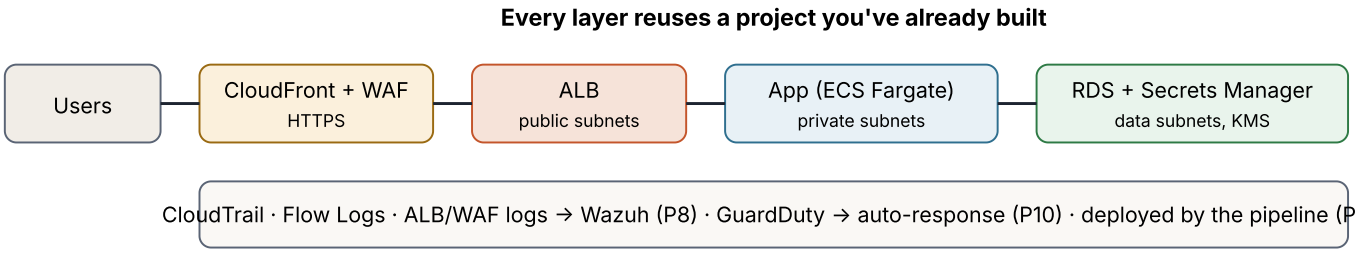
REMEMBER

Talking point: "I've used Sentinel as a multi-cloud SIEM with AWS and Azure data, and I can write KQL. The detection logic carries over from my Wazuh rules."

★ Capstone: a secure 3-tier app with detection and response

Month 6 · All tracks · Weeks 22, 24–25 · Folder `capstone/`

> **Objective: one project that shows the whole job: design securely, deploy through a secure pipeline, monitor centrally, detect an attack, respond automatically and by hand, and report it.**



1. **Network tier:** the VPC module with public, private-app and private-data subnets in two AZs; VPC endpoints for S3, ECR, logs and Secrets Manager, so the app needs no NAT gateway; Flow Logs on.
2. **Data tier:** RDS (smallest size) in the data subnets, encrypted with your KMS key, not publicly accessible; the password in Secrets Manager; its security group allows the database port **only from the app's security group**.
3. **App and front door:** a simple container app on ECS Fargate (from ECR, scanned on push) with a task role that can read only its own secret; an ALB whose security group accepts traffic only from CloudFront's managed prefix list; CloudFront with HTTPS and the WAF from P3.
4. **Security operations wired in:** CloudTrail, Flow Logs and ALB/WAF logs reach Wazuh; P8 rules and P10 automations cover this account; everything is deployed through the P12 pipeline. Write a STRIDE threat model of the design (done in week 22).
5. **The incident** (all in your own account): a multi-stage scenario: (a) a developer key is "leaked" and used; (b) the attacker opens the app security group to the world; (c) the attacker creates a new IAM user for persistence. Detect each stage, contain, investigate with a single timeline, recover from code.
6. **Deliver:** a README (architecture, security decisions, threat model, cost per hour, how to deploy and destroy), the incident report, and a **3–5 minute demo video**.

REMEMBER

Cost: RDS, ALB and Fargate all bill by the hour. Build, test and destroy in focused sessions, and keep a running total in the README.
Talking point: this is the project you open every interview with.

T Templates

T1 · Alert triage note (daily alert, two to ten lines)

```
Alert:      [name / rule ID]          Time (UTC): [ ]     Severity: [ ]
What:       [one sentence]
Who/where:  user [ ], host [ ], source IP [ ] (internal? reputation?)
Evidence:   [the 2–3 log lines or fields that matter]
Context:    normal for this user/host?  related alerts?  any success after failures?
Verdict:    TRUE POSITIVE / FALSE POSITIVE / BENIGN TRUE POSITIVE
Action:     close / escalate to Tier 2 / contain [what]      MITRE: [ID]
```

T2 · Detection write-up

```
Name and ID · Hypothesis (what attacker behaviour, MITRE ID) · Data source and fields
Logic (the rule) · True-positive test (steps + result) · False-positive test (steps +
result)
Known blind spots · Tuning notes · Response guidance for the analyst
```

T3 · Incident report

```
1. Summary (3–5 lines for a manager: what, impact, status)
2. Timeline (UTC): detection → analysis → containment → eradication → recovery
3. Scope and impact: systems, data, accounts affected
4. Root cause
5. Actions taken (with who and when)
6. Evidence (log excerpts, screenshots, hashes; where it is stored)
7. Lessons learned and recommendations (owner + due date for each)
```

T4 · Playbook

```
Trigger (which alerts) · Severity guide · Triage questions · Containment steps
Evidence to collect (before changing anything) · Eradication and recovery · Communication
(who to tell, when)
Closure criteria · Related detections and MITRE IDs
```

T5 · Assessment / posture report

```
Executive summary (plain language) · Scope and method (tools, standards, dates)
Score before → after · Findings table: ID, title, severity, affected resources, risk, fix,
status
Accepted risks (reason, owner) · Prioritised recommendations · Appendix: evidence
```

T6 · Project README

```
Title · Summary (3 lines) · Architecture diagram · What I built · Security decisions
(control → risk)
How I tested it · How I'd detect an attack on it · Cost and how to destroy · Lessons ·
Screenshots (redacted)
```

C Checklists and command reference

End-of-session cost checklist

- ☐ `terraform destroy` run for everything built today (or resources deleted by hand)
- ☐ No NAT gateways, load balancers, RDS instances, interface endpoints or WAF web ACLs left running
- ☐ No running EC2 instances or ECS tasks you don't need
- ☐ Billing page checked: nothing unexpected today
- ☐ Lab VMs shut down cleanly (and snapshotted after big changes)

Before every push

- ☐ Screenshots redacted: account IDs, emails, key IDs, public IPs, usernames, hostnames
- ☐ No secrets, `.tfstate`, `.tfvars` or credentials files staged (`git status`, `git diff --cached`)
- ☐ `git pull --rebase` first
- ☐ README updated; progress log updated

Commands you'll use every week

Task	Command
Who am I in AWS?	`aws sts get-caller-identity`
Refresh SSO login	`aws sso login --profile lab`
Recent CloudTrail events	`aws cloudtrail lookup-events --max-results 10`
What's running?	`aws ec2 describe-instances --query 'Reservations[].Instances[].[InstanceId,State.Name]' --output table`
Connect without SSH	`aws ssm start-session --target i-XXXX`
Terraform cycle	`terraform init` → `plan` → `apply` → `destroy`
Wazuh manager status/log	`sudo systemctl status wazuh-manager`; `sudo tail -f /var/ossec/logs/ossec.log`
Is syslog arriving?	`sudo tcpdump -ni any udp port 514 -c 5`
Test a Wazuh rule	Dashboard → Ruleset Test (or `/var/ossec/bin/wazuh-logtest`)
Windows: refresh policy	`gpupdate /force`
Windows: Wazuh agent restart	`NET STOP WazuhSvc` then `NET START WazuhSvc`
DNS records for email	`dig +short TXT domain`; `dig +short TXT _dmarc.domain`

Q Interview questions by month

Answer each one out loud, then write your best answer in `notes/interview-bank.md`. Bring them to review sessions with Claude for interview-style grading.

Month	Questions
1	What are the first five things you do in a new AWS account? · Walk me through how a request reaches an instance in a public subnet. · Security groups vs NACLs? · How do you triage an alert? · How would you detect a port scan?
2	A bucket was made public to host a site. How do you fix it without breaking the site? · How does KMS envelope encryption work? · Show me how you'd spot SQL injection in logs. · Walk me through a phishing investigation. · What do SPF, DKIM and DMARC each do?
3	401 vs 403? · How does a JWT authorizer decide? · How does AWS evaluate an SCP, a boundary and an identity policy? · Which Windows event IDs do you check after a suspected brute force? · Why enable PowerShell script block logging?
4	Which CloudTrail events would you alert on first? · Your SIEM shows nothing for 20 minutes after an attack. Why might that be? · How do you prioritise 300 Security Hub findings? · What is threat hunting, and how is it different from alerting? · How do you measure detection coverage?
5	Walk me through the NIST incident response lifecycle. · An access key leaked. What do you do in the first 15 minutes? · How do you contain an EC2 instance without destroying evidence? · Why might swapping security groups not cut an attacker's connection? · What goes in an incident report?
6	Why use OIDC in CI/CD instead of access keys? · What does Checkov catch, and what can't it catch? · Write a KQL query for console logins without MFA. · Tell me about your capstone: decisions, trade-offs, cost. · Tell me about a time you were wrong and how you found out (Case 001).

REMEMBER

Only test systems you own or have written permission to test. Every simulation in this plan runs in your own home lab or your own AWS and Azure accounts.